LECTURE 19 · DISCRETE MATHEMATICS

From sequences to recurrences

Lecture 18 found a term and a total from a formula. Now let earlier values define the next one. How much work does this recursive rule describe?

$$T(n) = 2T(n/2) + n$$



A recursive definition needs a rule and a base

A recurrence defines a new value using earlier values. Initial conditions start the process.

Well founded

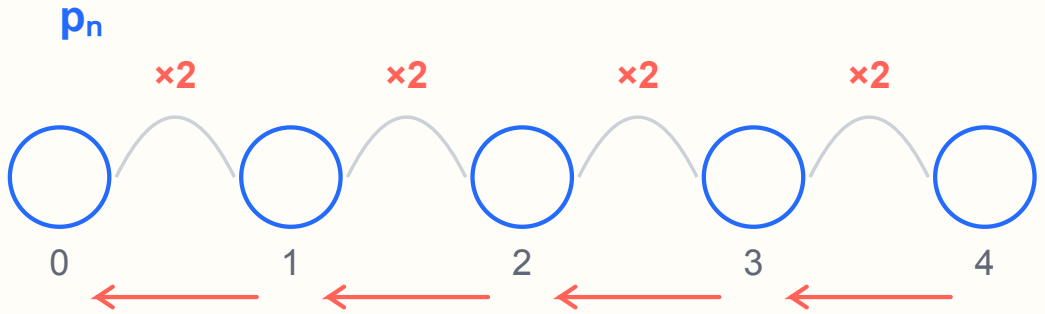
$$p_0 = 1, \quad p_n = 2p_{n-1} \quad (n \geq 1)$$

Incomplete

$p_n = 2p_{n-1}$ **without a value of** p_0

Uninformative self-reference

$p_n = p_n$ **supplies no value for a new term.**



$$p_4 \to p_3 \to p_2 \to p_1 \to p_0$$

A recursive definition needs a rule and a base

A recurrence defines a new value using earlier values. Initial conditions start the process.

Well founded

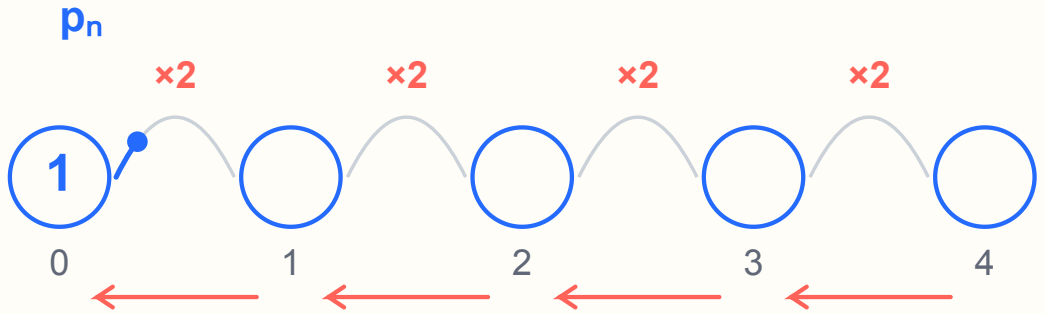
$$p_0 = 1, \quad p_n = 2p_{n-1} \quad (n \geq 1)$$

Incomplete

$p_n = 2p_{n-1}$ **without a value of** p_0

Uninformative self-reference

$p_n = p_n$ **supplies no value for a new term.**



$p_0 = 1$

A recursive definition needs a rule and a base

A recurrence defines a new value using earlier values. Initial conditions start the process.

Well founded

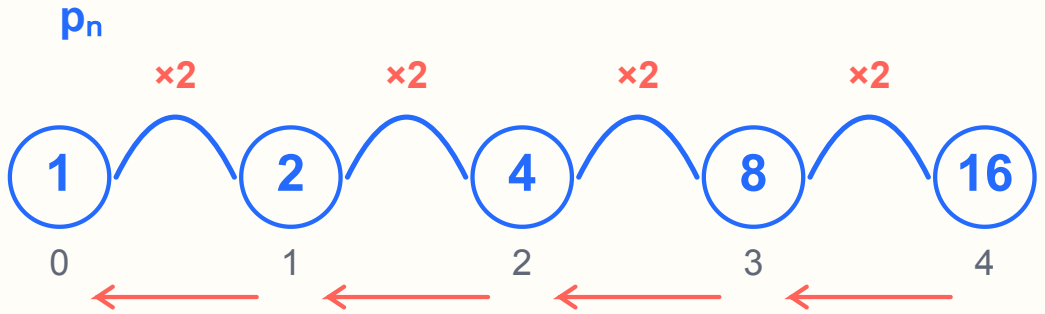
$$p_0 = 1, \quad p_n = 2p_{n-1} \quad (n \geq 1)$$

Incomplete

$p_n = 2p_{n-1}$ **without a value of** p_0

Uninformative self-reference

$p_n = p_n$ **supplies no value for a new term.**



$$p_4 = 2^4 p_0 = 16$$

A second-order rule uses two previous terms



$F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Two initial values

$$F_0 = 0, \quad F_1 = 1$$

Then compute

$$0, 1, 1, 2, 3, 5, 8, \ldots$$

$F_0 = 0, \quad F_1 = 1$

A second-order rule uses two previous terms

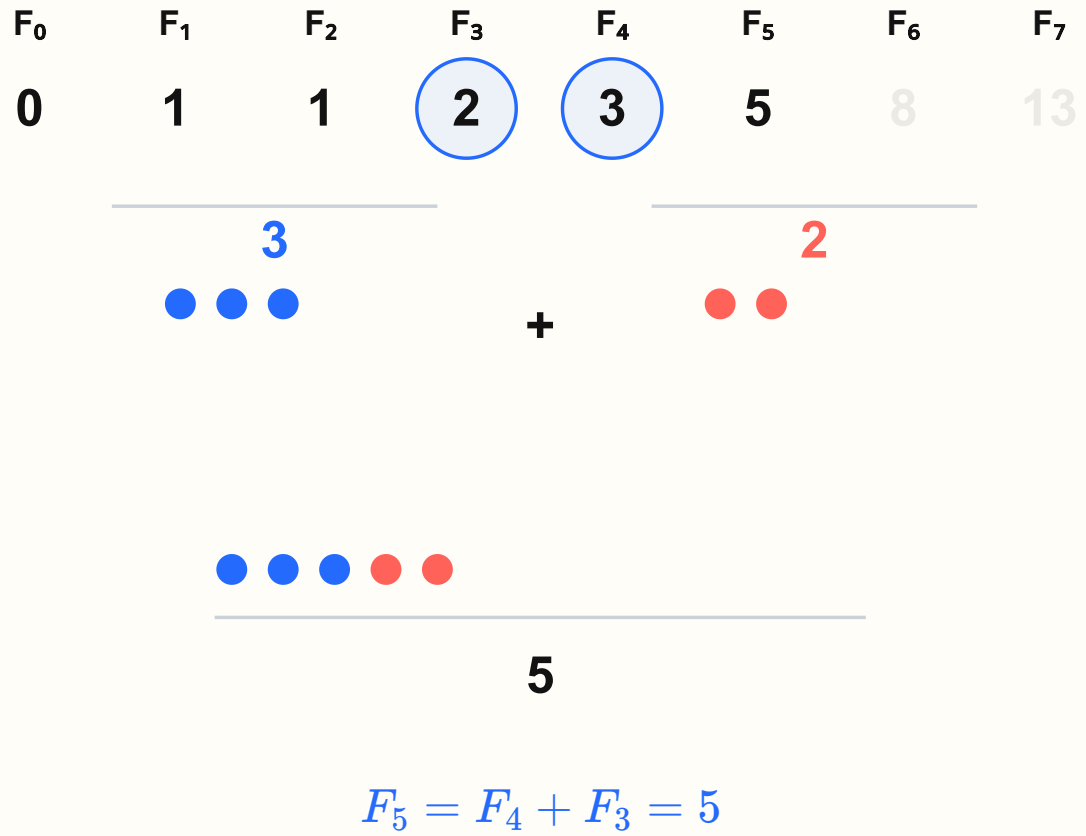
$F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Two initial values

$F_0 = 0, \quad F_1 = 1$

Then compute

$0, 1, 1, 2, 3, 5, 8, \ldots$



$F_5 = F_4 + F_3 = 5$

A second-order rule uses two previous terms

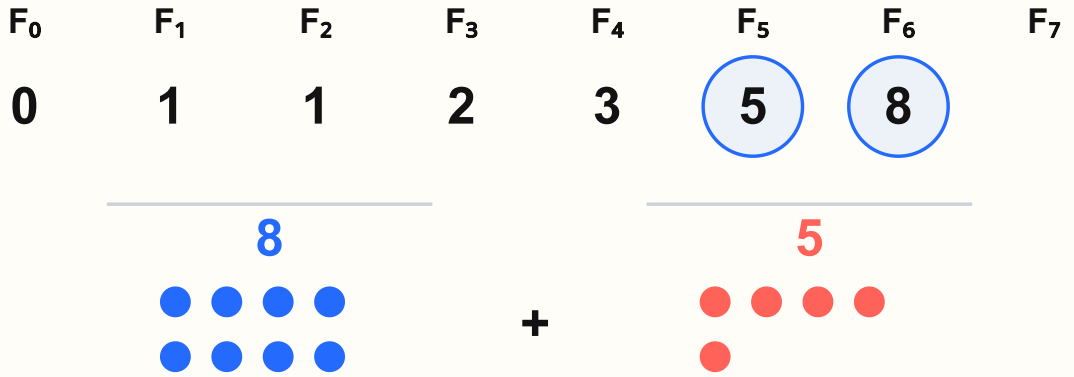
$F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Two initial values

$F_0 = 0, \quad F_1 = 1$

Then compute

$0, 1, 1, 2, 3, 5, 8, \ldots$



$F_7 = 8 + 5 = 13$

A second-order rule uses two previous terms

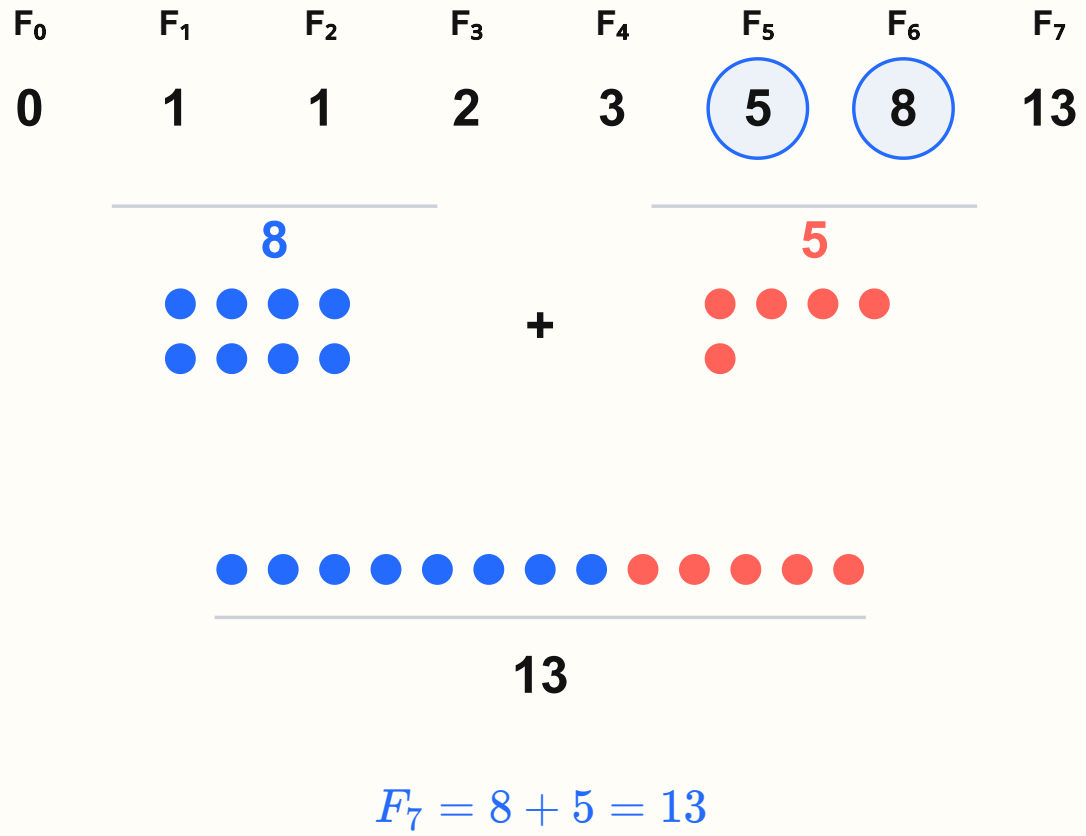
$F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Two initial values

$F_0 = 0, \quad F_1 = 1$

Then compute

$0, 1, 1, 2, 3, 5, 8, \ldots$



$F_7 = 8 + 5 = 13$

Count staircase routes by their final move

Use moves of 1 or 2 steps. Let s_n count ordered routes to step n.

01 **What are s_0 and s_1?**

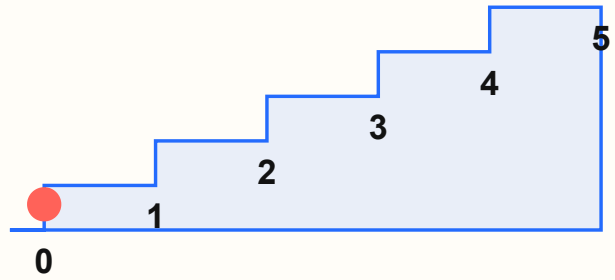
$s_0 = 1$ **(the empty route),** $s_1 = 1$.

02 **Separate routes by their final move.**

$s_n = s_{n-1} + s_{n-2}$ **for** $n \geq 2$.

03 **Find s_0 through s_5.**

1, 1, 2, 3, 5, 8.



Last move 1 5 routes **Last move 2**

1	1	1	1	1
2		1	1	1
1	2		1	1
1	1	2		1
2		2		1

$$s_0 = 1, \quad s_1 = 1$$

Count staircase routes by their final move

Use moves of 1 or 2 steps. Let s_n count ordered routes to step n.

01 **What are s_0 and s_1?**

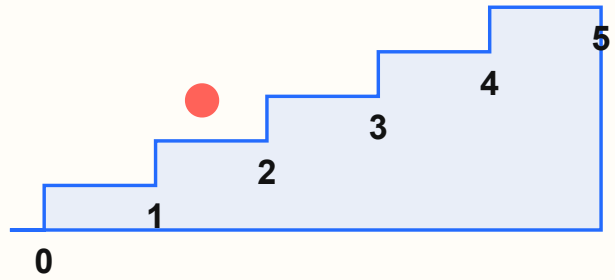
$s_0 = 1$ (the empty route), $s_1 = 1$.

02 **Separate routes by their final move.**

$s_n = s_{n-1} + s_{n-2}$ for $n \geq 2$.

03 **Find s_0 through s_5.**

1, 1, 2, 3, 5, 8.



Last move 1 5 routes

1	1	1	1	1
2	1	1	1	
1	2	1	1	
1	1	2	1	
2	2	1		

Last move 2 3 routes

1	1	1	2
2	1	2	
1	2	2	

$$s_5 = s_4 + s_3 = 5 + 3$$

Count staircase routes by their final move

Use moves of 1 or 2 steps. Let s_n count ordered routes to step n.

01 **What are s_0 and s_1?**

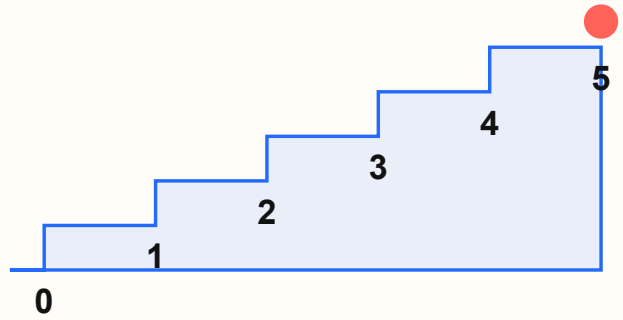
$s_0 = 1$ (the empty route), $s_1 = 1$.

02 **Separate routes by their final move.**

$s_n = s_{n-1} + s_{n-2}$ for $n \geq 2$.

03 **Find s_0 through s_5.**

1, 1, 2, 3, 5, 8.



Last move 1 5 routes

1	1	1	1	1
2	1	1	1	
1	2	1	1	
1	1	2	1	
2	2	1		

Last move 2 3 routes

1	1	1	2
2	1	2	
1	2	2	

$s_5 = 8$

Factorials have a recursive definition

$$f_0 = 1, \qquad f_n = n f_{n-1} \quad (n \geq 1)$$

One empty ordering

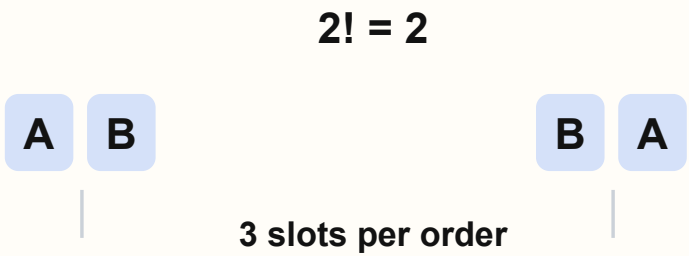
$$0! = 1$$

Insert the new symbol

$$n! = n(n-1)!$$

Three symbols

$$3! = 3 \cdot 2! = 6$$



$$2! = 2$$

Factorials have a recursive definition

$$f_0 = 1, \qquad f_n = n f_{n-1} \quad (n \geq 1)$$

One empty ordering

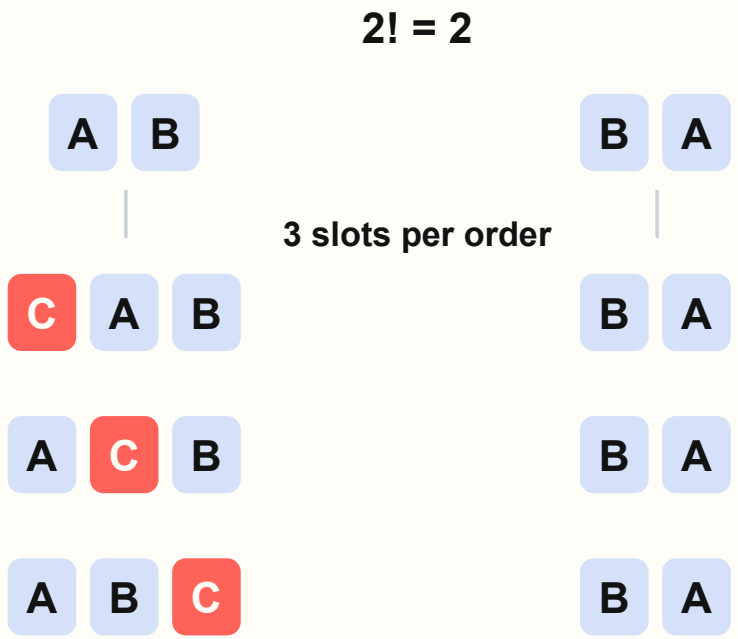
$$0! = 1$$

Insert the new symbol

$$n! = n(n-1)!$$

Three symbols

$$3! = 3 \cdot 2! = 6$$



3 slots $\times$ 2 orders

Factorials have a recursive definition

$$f_0 = 1, \qquad f_n = n f_{n-1} \quad (n \geq 1)$$

One empty ordering

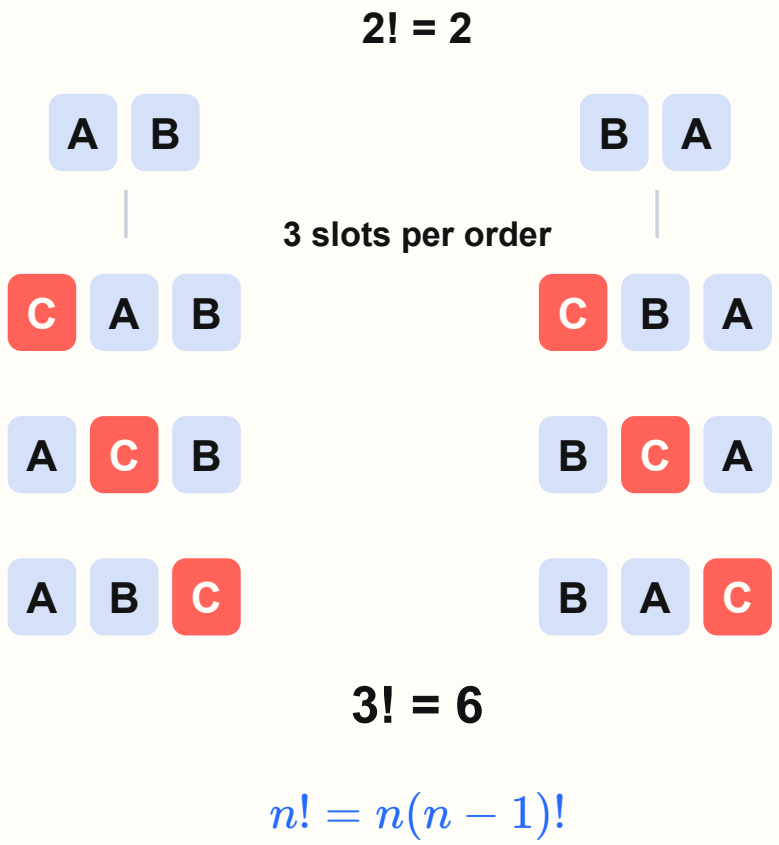
$$0! = 1$$

Insert the new symbol

$$n! = n(n-1)!$$

Three symbols

$$3! = 3 \cdot 2! = 6$$



$$n! = n(n-1)!$$

Population models balance gains and losses

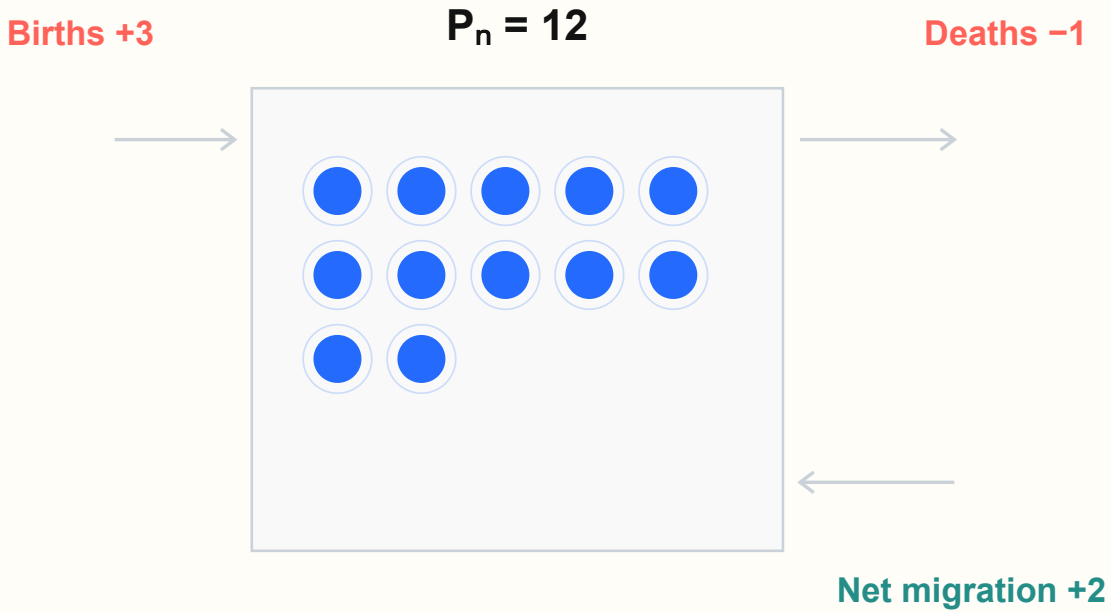
Start from P_0 and specify the gains and losses in each period.

Accounting identity

$$P_{n+1} = P_n + b_n - d_n + m_n$$

Required data

P_0 and the rules for b_n, d_n, m_n.



$$P_n = 12$$

b_n: births; d_n: deaths; m_n: net migration during period n.

Population models balance gains and losses

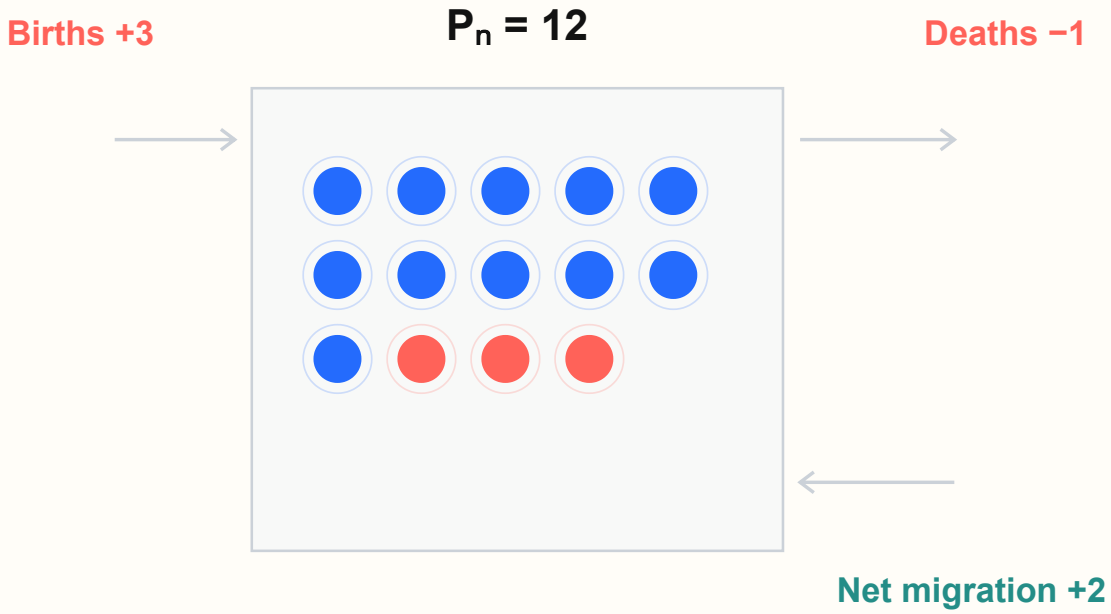
Start from P_0 and specify the gains and losses in each period.

Accounting identity

$$P_{n+1} = P_n + b_n - d_n + m_n$$

Required data

P_0 and the rules for b_n, d_n, m_n.



$$15 - 1 = 14$$

b_n: births; d_n: deaths; m_n: net migration during period n.

Population models balance gains and losses

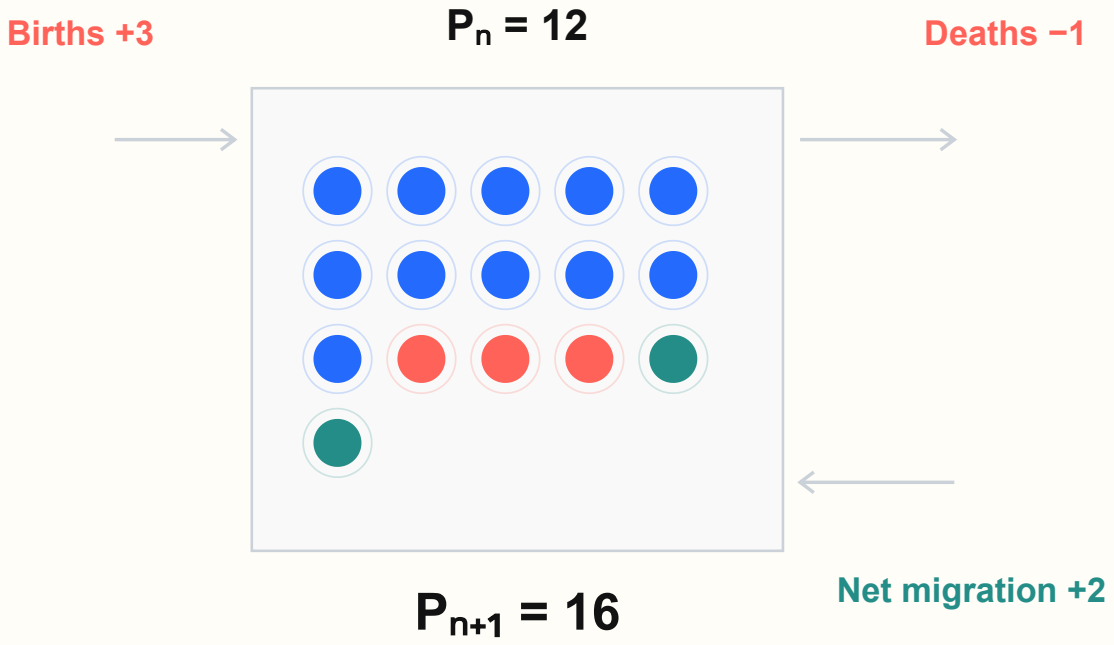
Start from P_0 and specify the gains and losses in each period.

Accounting identity

$$P_{n+1} = P_n + b_n - d_n + m_n$$

Required data

P_0 and the rules for b_n, d_n, m_n.



$$P_{n+1} = 12 + 3 - 1 + 2 = 16$$

b_n: births; d_n: deaths; m_n: net migration during period n.

A nonlinear recurrence can still be evaluated

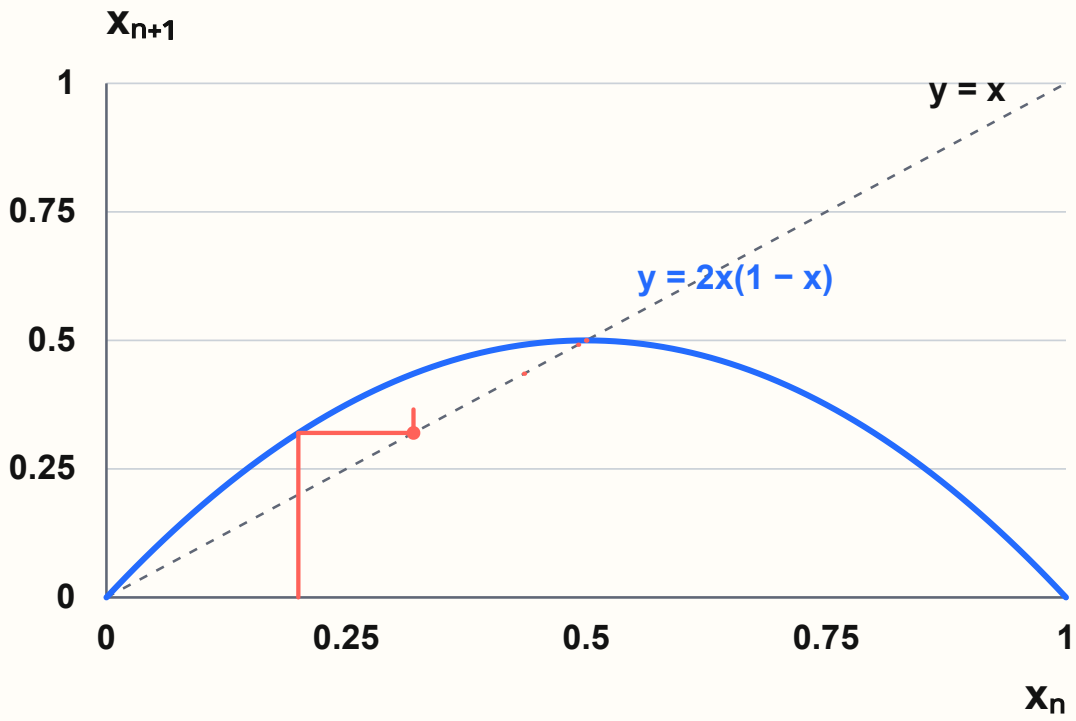
The logistic map is $x_{n+1} = r x_n (1 - x_n)$.

Choose the data

$$r = 2, \quad x_0 = 0.2$$

Compute carefully

$$x_1 = 0.32, \quad x_2 = 0.4352$$



$$x_1 = 2(0.2)(0.8) = 0.32$$

The selected orbit approaches $1/2$; other parameters can behave differently.

A nonlinear recurrence can still be evaluated

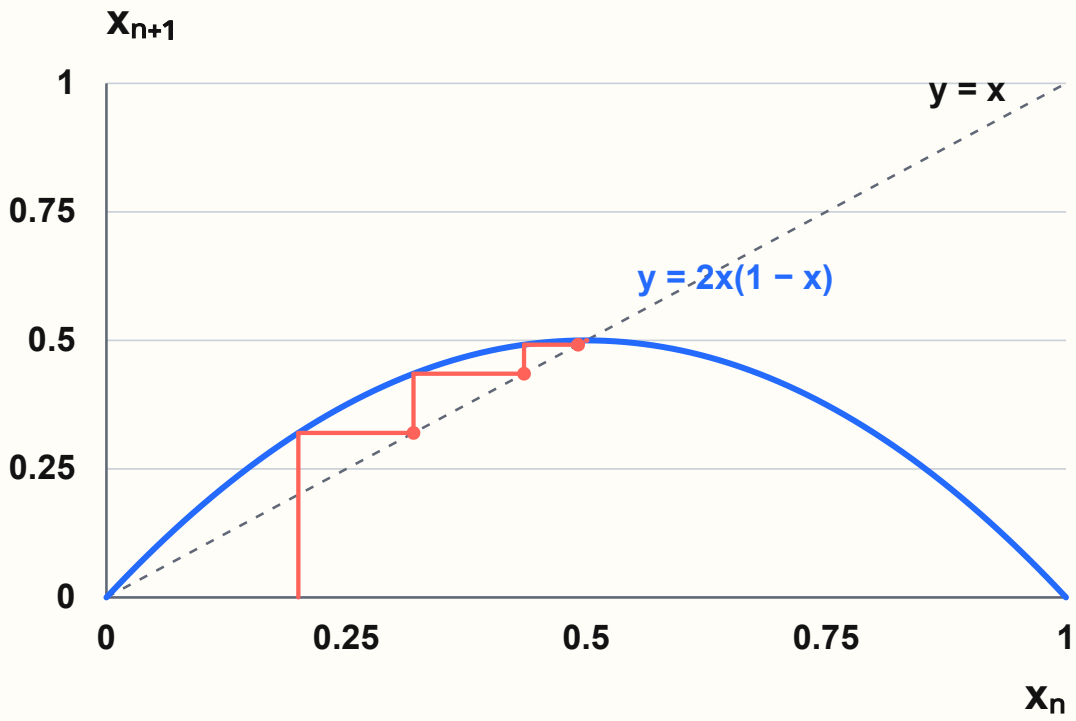
The logistic map is $x_{n+1} = r x_n (1 - x_n)$.

Choose the data

$$r = 2, \quad x_0 = 0.2$$

Compute carefully

$$x_1 = 0.32, \quad x_2 = 0.4352$$



$$x_3 = 0.49160192$$

The selected orbit approaches $1/2$; other parameters can behave differently.

A nonlinear recurrence can still be evaluated

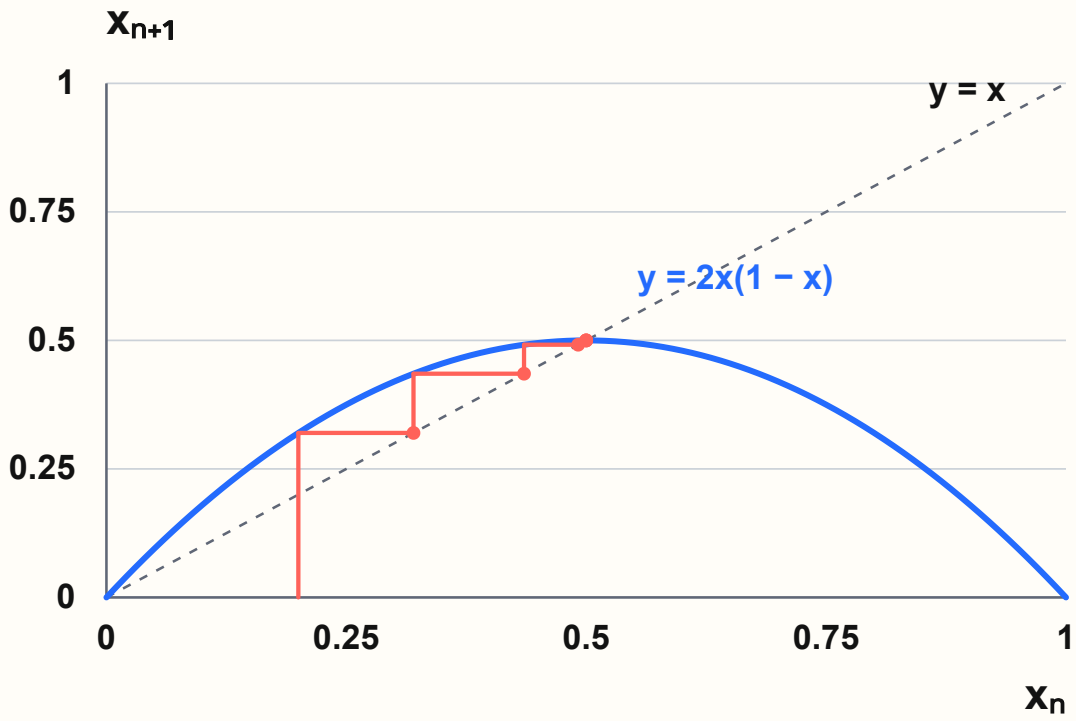
The logistic map is $x_{n+1} = r x_n (1 - x_n)$.

Choose the data

$$r = 2, \quad x_0 = 0.2$$

Compute carefully

$$x_1 = 0.32, \quad x_2 = 0.4352$$



$$x_n \longrightarrow \tfrac{1}{2} \quad (r = 2,\ x_0 = 0.2)$$

The selected orbit approaches $1/2$; other parameters can behave differently.

Change one number. Change the behavior

$$x_{n+1} = r x_n (1 - x_n)$$

Start every orbit at $x_0 = 0.2$.

$r = 2.9$ · one limiting value

$x_n \to 0.655\ldots$

$r = 3.2$ · a 2-cycle

$0.513\ldots \leftrightarrow 0.799\ldots$

$r = 3.5$ · a 4-cycle

Four values repeat in order.

$r = 4.0$ · chaos

Irregular fluctuations; no random input.



$r = 2.9$: a stable population.

Each plot shows generations 100–132, after the initial transient. Population is a fraction between 0 and 1.

Change one number. Change the behavior

$$x_{n+1} = r x_n (1 - x_n)$$

Start every orbit at $x_0 = 0.2$.

$r = 2.9$ · one limiting value

$x_n \to 0.655\ldots$

$r = 3.2$ · a 2-cycle

$0.513\ldots \leftrightarrow 0.799\ldots$

$r = 3.5$ · a 4-cycle

Four values repeat in order.

$r = 4.0$ · chaos

Irregular fluctuations; no random input.



$r = 3.2$: two values alternate.

Each plot shows generations 100–132, after the initial transient. Population is a fraction between 0 and 1.

Change one number. Change the behavior

$$x_{n+1} = r x_n (1 - x_n)$$

Start every orbit at $x_0 = 0.2$.

$r = 2.9$ · one limiting value

$x_n \to 0.655\ldots$

$r = 3.2$ · a 2-cycle

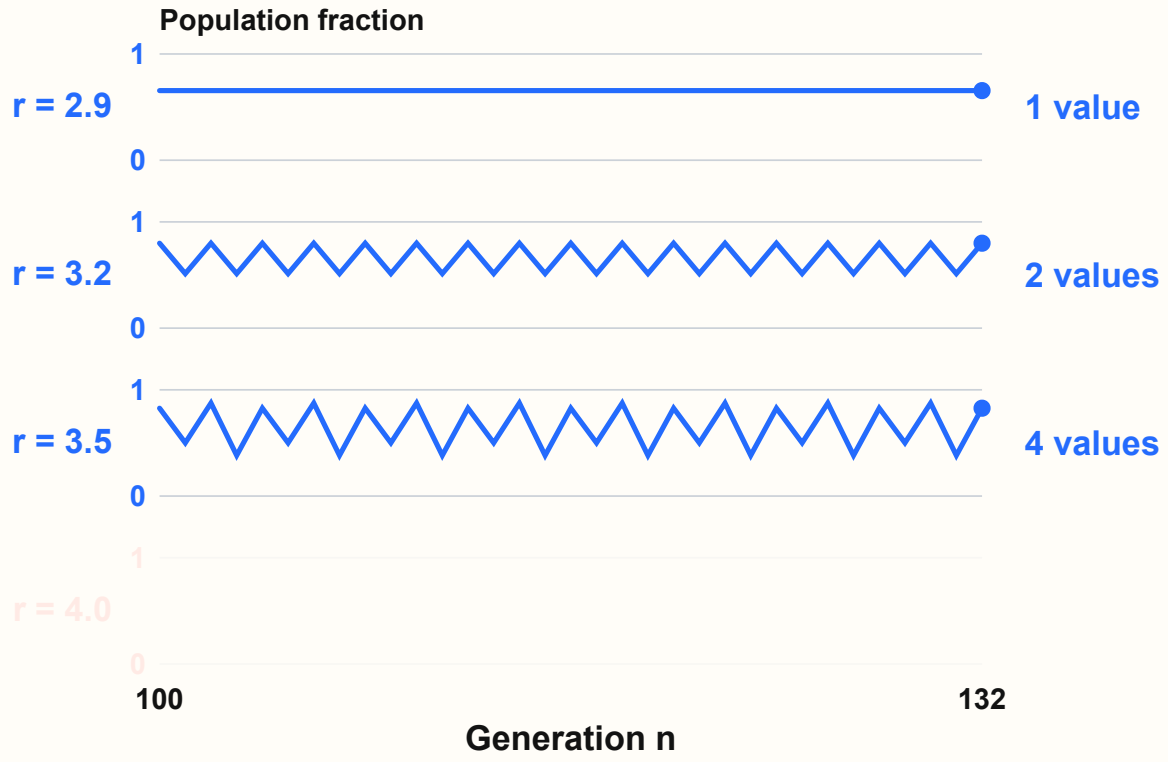
$0.513\ldots \leftrightarrow 0.799\ldots$

$r = 3.5$ · a 4-cycle

Four values repeat in order.

$r = 4.0$ · chaos

Irregular fluctuations; no random input.



$r = 3.5$: four values repeat.

Each plot shows generations 100–132, after the initial transient. Population is a fraction between 0 and 1.

Change one number. Change the behavior

$$x_{n+1} = r x_n (1 - x_n)$$

Start every orbit at $x_0 = 0.2$.

$r = 2.9$ · one limiting value

$x_n \to 0.655\ldots$

$r = 3.2$ · a 2-cycle

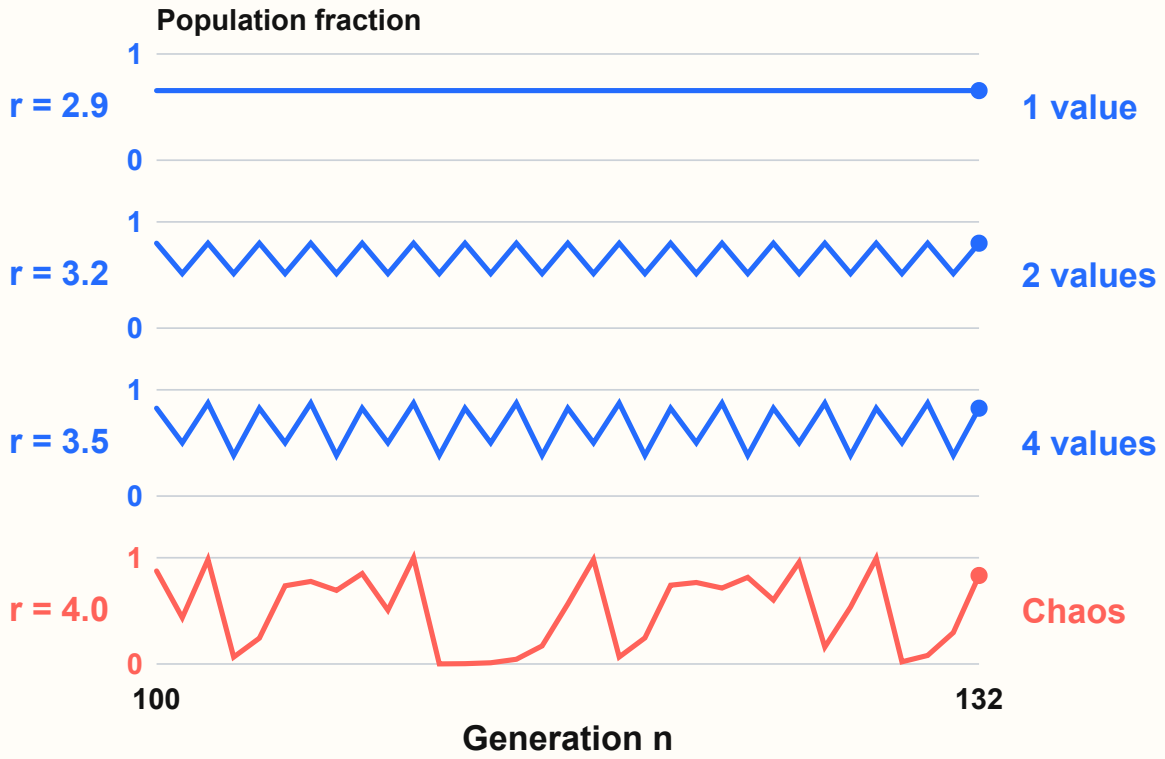
$0.513\ldots \leftrightarrow 0.799\ldots$

$r = 3.5$ · a 4-cycle

Four values repeat in order.

$r = 4.0$ · chaos

Irregular fluctuations; no random input.



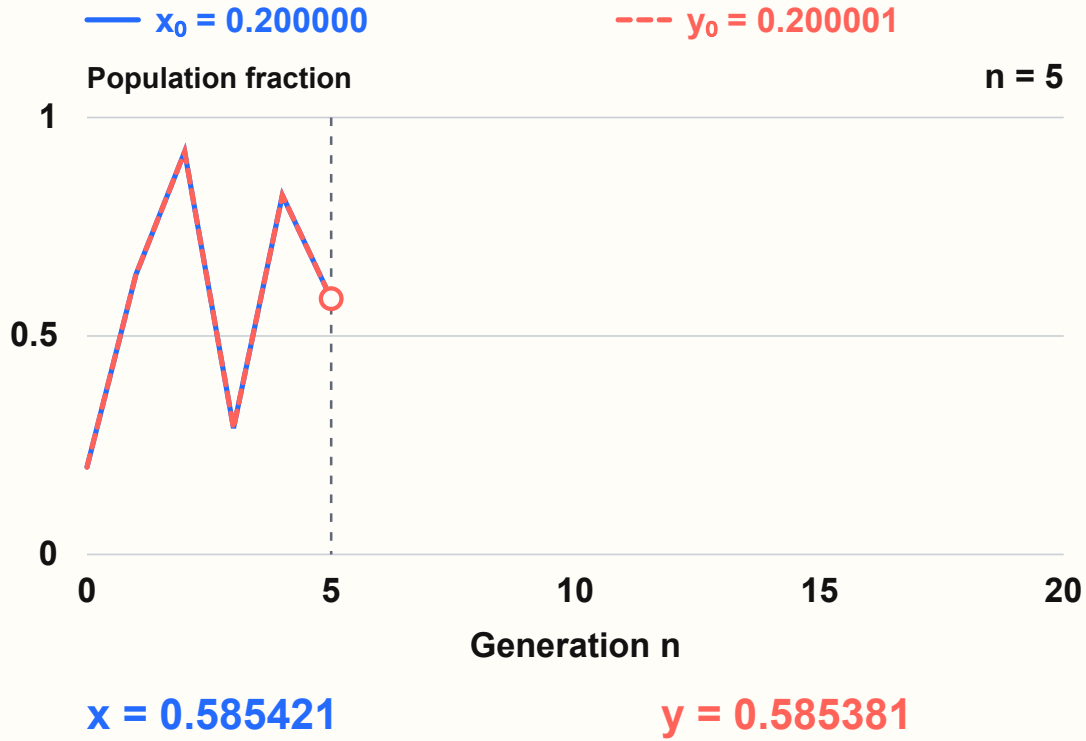
$r = 4$: a deterministic rule, chaotic behavior.

Each plot shows generations 100–132, after the initial transient. Population is a fraction between 0 and 1.

One millionth apart. Then worlds apart

$r = 4$: use the same rule for two almost identical starting populations.

n	x_n	y_n
0	0.200000	0.200001
5	0.585421	0.585381
10	0.147837	0.148746
20	0.820014	0.031369



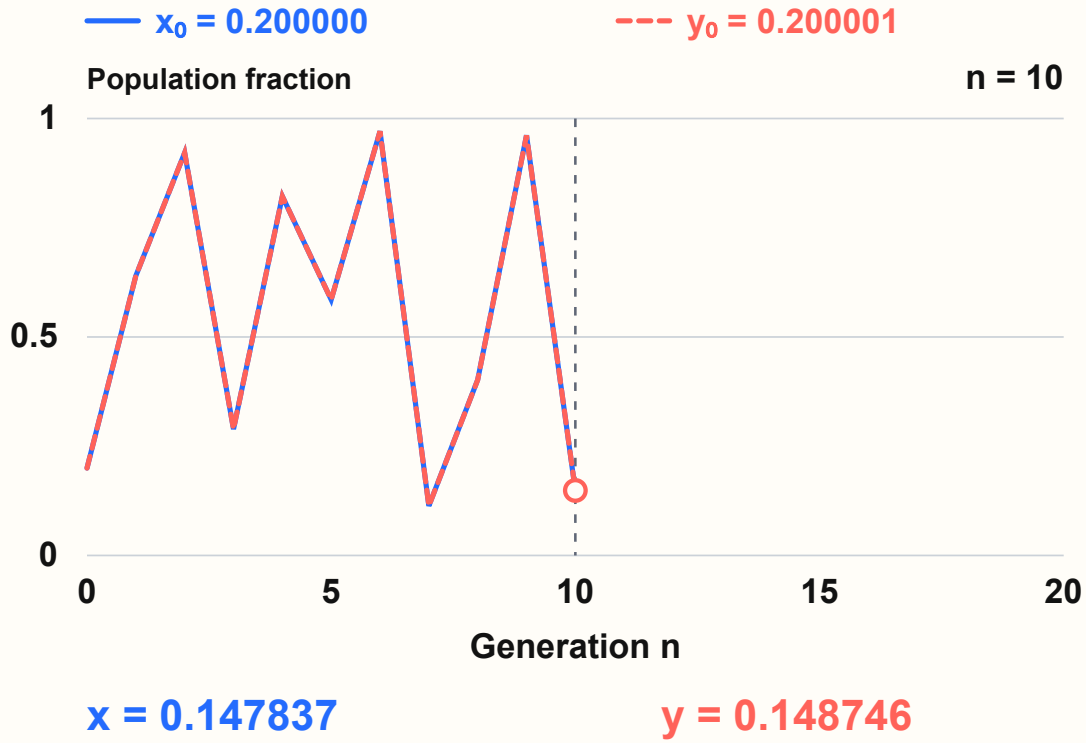
Generation 5: still almost indistinguishable.

After 20 generations, the gap is about 0.79. Displayed values are rounded only after iteration.

One millionth apart. Then worlds apart

$r = 4$: use the same rule for two almost identical starting populations.

n	x_n	y_n
0	0.200000	0.200001
5	0.585421	0.585381
10	0.147837	0.148746
20	0.820014	0.031369



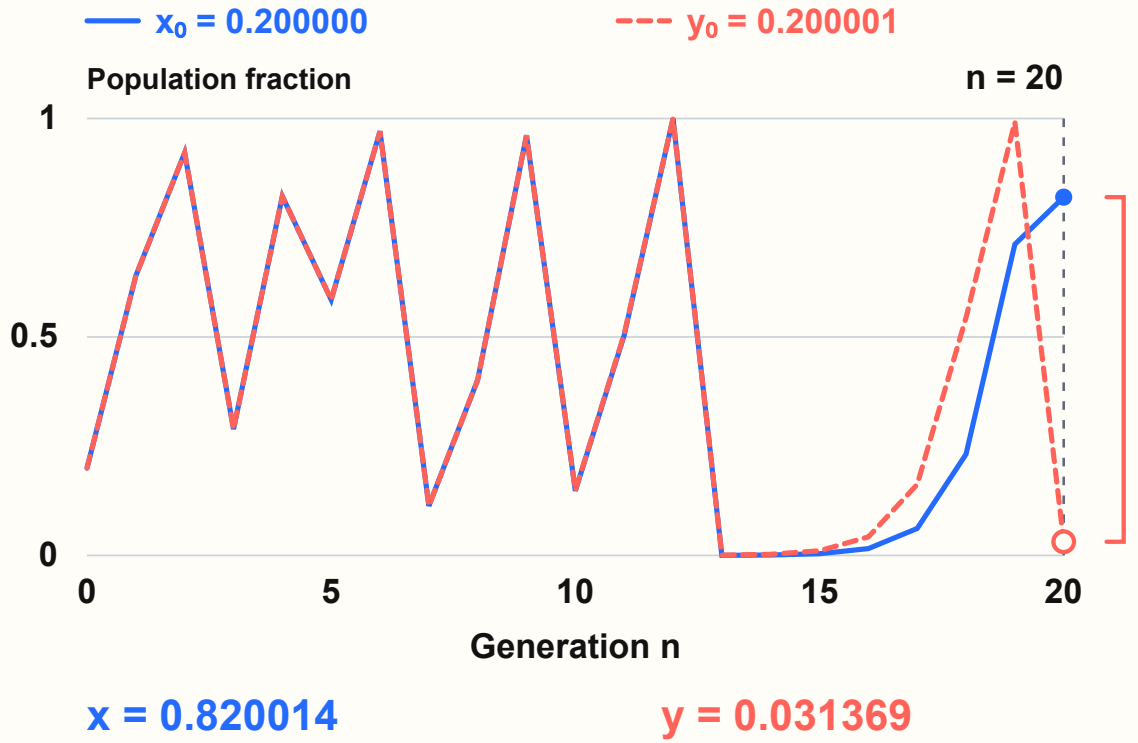
Generation 10: the gap is only 0.000910.

After 20 generations, the gap is about 0.79. Displayed values are rounded only after iteration.

One millionth apart. Then worlds apart

$r = 4$: use the same rule for two almost identical starting populations.

n	x_n	y_n
0	0.200000	0.200001
5	0.585421	0.585381
10	0.147837	0.148746
20	0.820014	0.031369



Generation 20: $|x_{20} - y_{20}| \approx 0.788644.$

After 20 generations, the gap is about 0.79. Displayed values are rounded only after iteration.

Can a deterministic future be unpredictable?

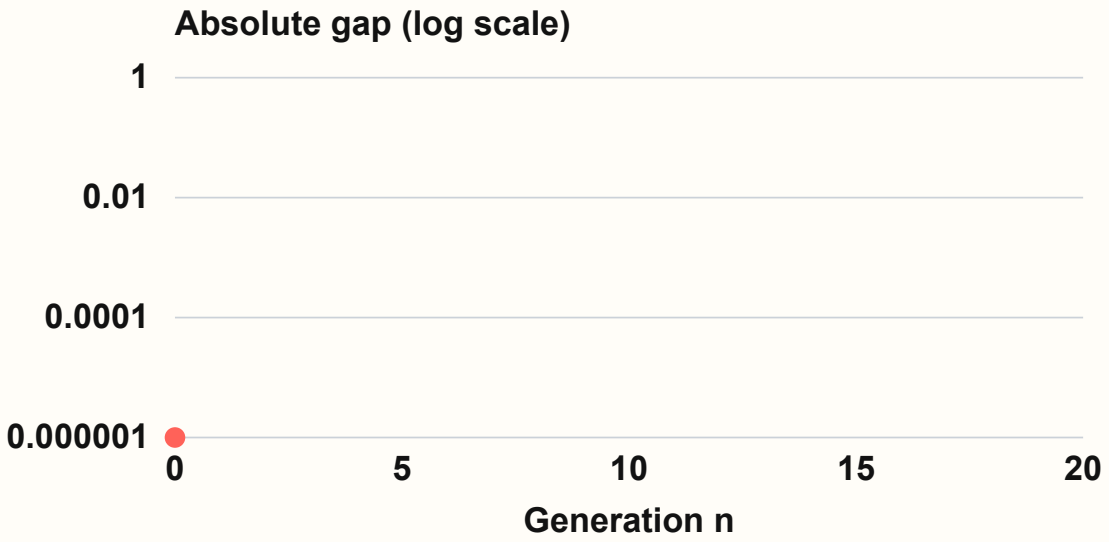
If a system follows a perfectly deterministic formula, can its future still be practically unpredictable?

No randomness

Every next value is fixed by the current value.

Limited precision

A tiny uncertainty in the starting value can become a large uncertainty later.



Predictable rule. How predictable is the outcome?

This sensitivity to initial conditions is a hallmark of deterministic chaos.

Can a deterministic future be unpredictable?

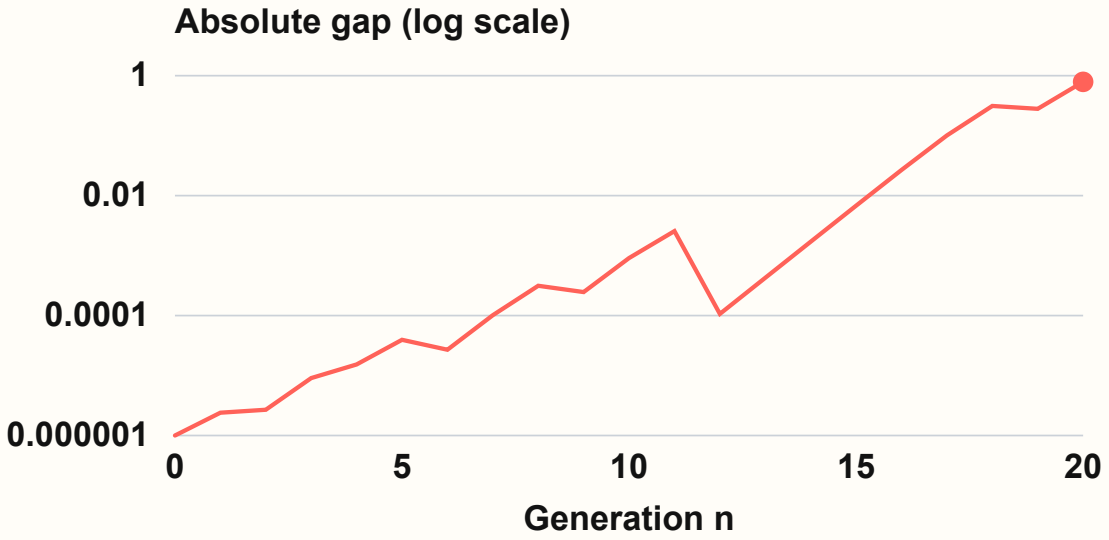
If a system follows a perfectly deterministic formula, can its future still be practically unpredictable?

No randomness

Every next value is fixed by the current value.

Limited precision

A tiny uncertainty in the starting value can become a large uncertainty later.



Yes. Deterministic chaos.

0.000001 → 0.788644

About 789,000 times the initial gap

No randomness was added at any step.

This sensitivity to initial conditions is a hallmark of deterministic chaos.

The power and limits of recurrences

A short mathematical definition need not give us a way to compute its values.

Power · simulate any algorithm

$$S_{t+1} = F(S_t)$$

Let S_t encode the full program state. A computable update F performs the next step. With enough state, recurrences can describe any algorithm's evolution.

Limit · the halting sequence

$$P_1, P_2, P_3, \ldots$$

$$a_n = \begin{cases} 1 & \text{if } P_n(n) \text{ halts,} \\ 0 & \text{if } P_n(n) \text{ runs forever.} \end{cases}$$

List all programs in a fixed, effective order. $P_n(n)$ means program P_n run on input n.

Why no computable recurrence can generate it

Computable initial data + a rule that finishes each step would let us compute any a_n. We could then decide whether $P_n(n)$ halts for every n—impossible by the halting theorem (Turing, 1936).

Chaos is computable step by step. The halting sequence is well defined but not computable.

Two recursive calls, then a merge

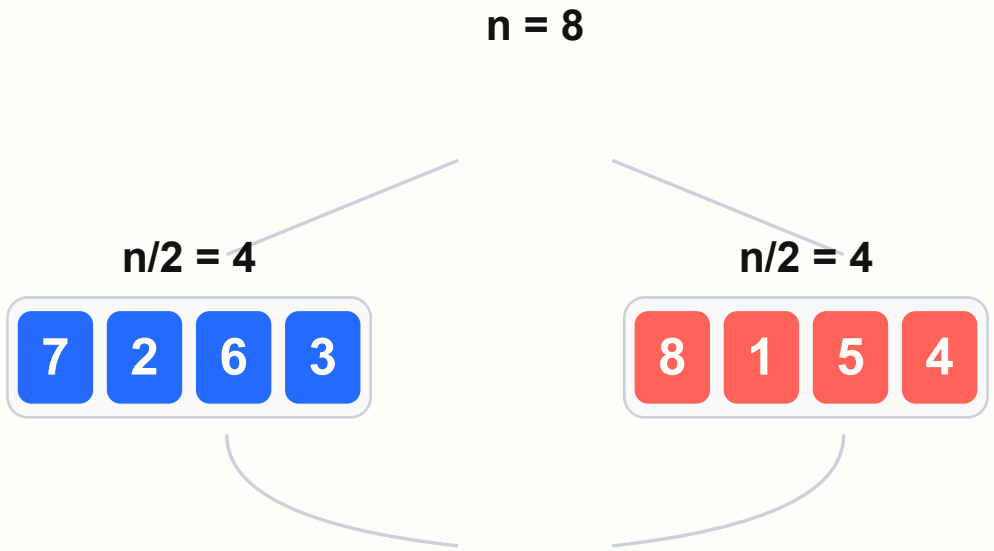
Split a list into two halves, sort each recursively, then merge.

Work model, $n \geq 2$ a power of 2

$$T(n) = 2T(n/2) + n$$

Base case

$$T(1) = c$$



$2T(4)$

Here n is a merge-work allowance, not the exact comparison count.

Two recursive calls, then a merge

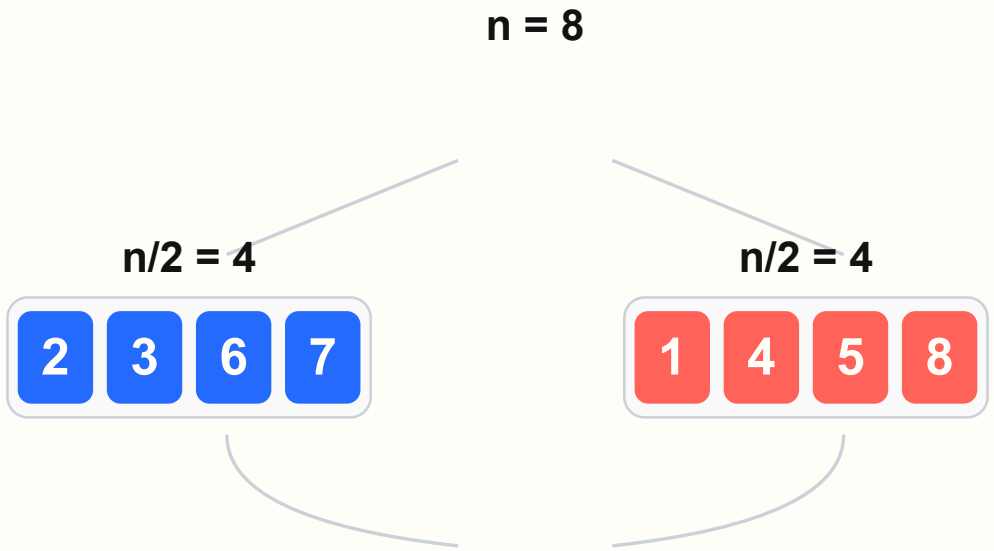
Split a list into two halves, sort each recursively, then merge.

Work model, $n \geq 2$ a power of 2

$$T(n) = 2T(n/2) + n$$

Base case

$$T(1) = c$$



$2T(4)$

Here n is a merge-work allowance, not the exact comparison count.

Two recursive calls, then a merge

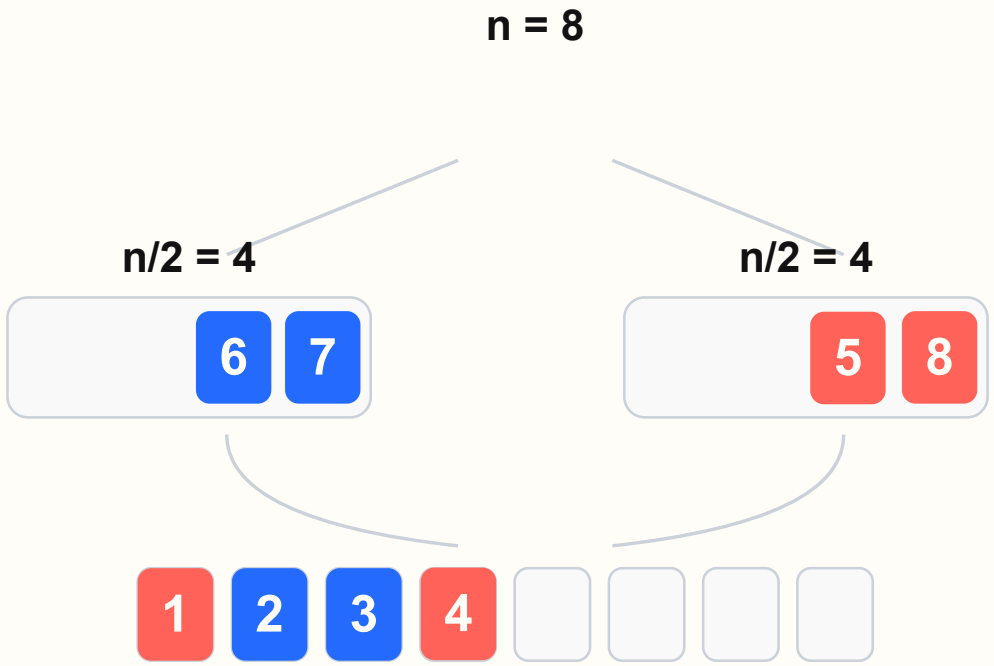
Split a list into two halves, sort each recursively, then merge.

Work model, $n \geq 2$ a power of 2

$$T(n) = 2T(n/2) + n$$

Base case

$$T(1) = c$$



$$T(8) = 2T(4) + 8$$

Here n is a merge-work allowance, not the exact comparison count.

Two recursive calls, then a merge

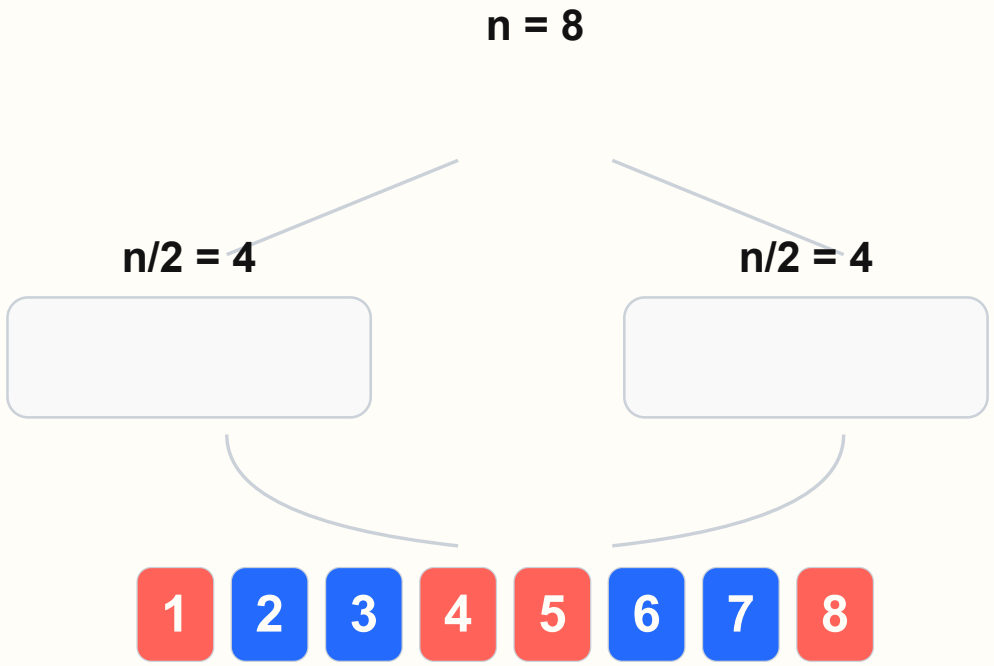
Split a list into two halves, sort each recursively, then merge.

Work model, $n \geq 2$ a power of 2

$$T(n) = 2T(n/2) + n$$

Base case

$$T(1) = c$$



$$T(n) = 2T(n/2) + n$$

Here n is a merge-work allowance, not the exact comparison count.

A tree records the work of every call

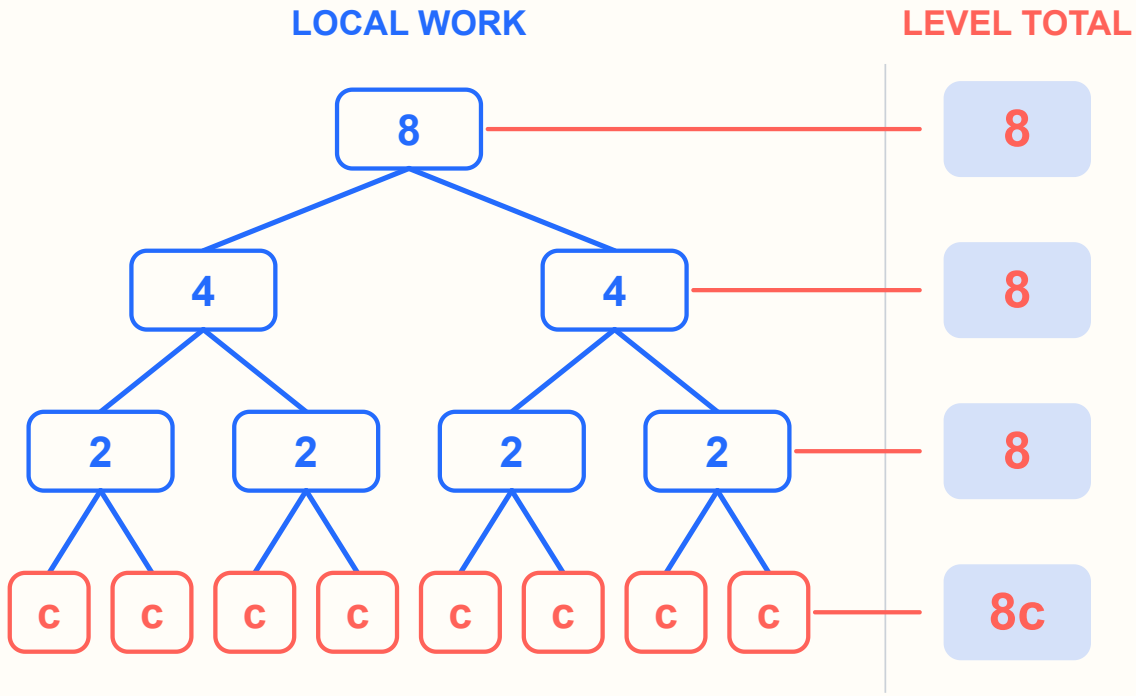
$T(n) = 2T(n/2) + n$, $T(1) = c$. Start with $n = 8$.

Inside each node

Charge only the work done in that call.

At the bottom

Eight size-1 calls each cost c.



$$T(8) = 8 + 8 + 8 + 8c$$

Adding $T(m)$ at every node would count descendants more than once.

Every internal level costs the same amount

The picture shows the first three internal levels for $n \geq 8$.

Number of calls

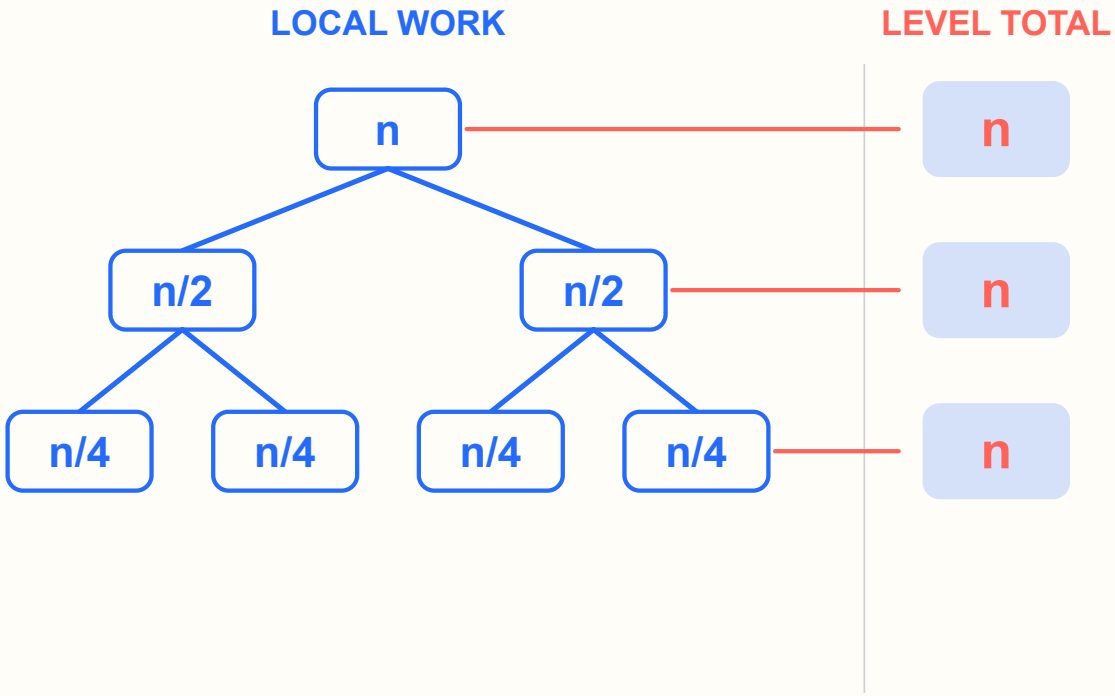
$$2^i$$

Work per call

$$n/2^i$$

Work across the level

$$2^i \cdot (n/2^i) = n$$



At depth i: $2^i \cdot (n/2^i) = n$.

The number of calls doubles while the work per call halves.

The leaves add a separate base-case cost

The recursion stops when $n/2^k = 1$, so $k = \log_2 n$.

Internal levels

$0, 1, \ldots, k-1$: **exactly k levels**

Leaf level

$2^k = n$ **leaves, each costing** c

INTERNAL WORK

$$\underbrace{n + n + \cdots + n}_{k \text{ levels}} = kn$$

LEAF WORK

$$2^k T(1) = nc$$

TOTAL

$$T(n) = kn + nc$$

USE $k = \log_2 n$

$$T(n) = n \log_2 n + cn$$

Internal levels plus leaves account for every call.

$T(n) = n \log_2 n + cn$. **For** $n = 8$: $T(8) = 24 + 8c$.

EXERCISE 2

Your turn: quadratic work in each call

$U(n) = 2U(n/2) + n^2$, $U(1) = 1$, with $n = 2^k$, $k \geq 0$.

01 **What does internal depth $i < k$ cost?**

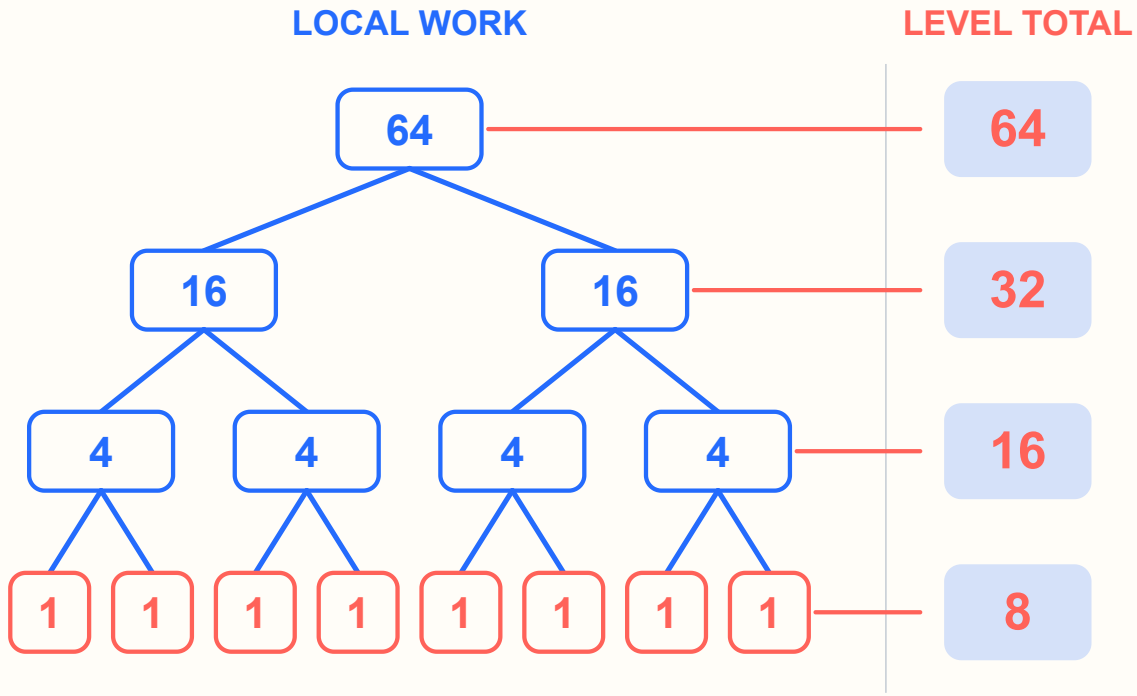
$2^i(n/2^i)^2 = n^2/2^i$.

02 **Sum internal work and leaf work separately.**

$n^2 \sum_{i=0}^{k-1} 2^{-i} = 2n^2 - 2n$; **leaves contribute n.**

03 **Find $U(n)$ and check $n = 1, 2$.**

$U(n) = 2n^2 - n$; $U(1) = 1, U(2) = 6$.



$$U(8) = 64 + 32 + 16 + 8 = 120$$

Use Lecture 18's finite geometric sum; stop before the leaf level.

One recursive call gives one path

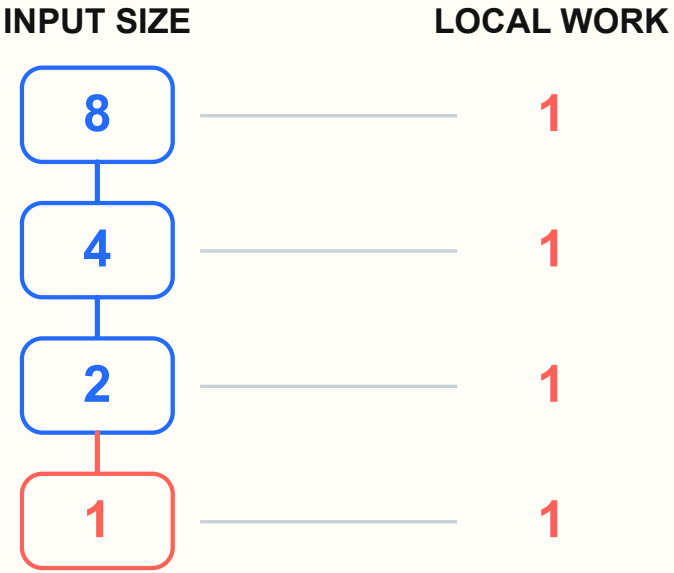
$B(n) = B(n/2) + 1$, $B(1) = 1$, with $n = 2^k$.

One call at each depth

$$B(n) = B(n/2^j) + j$$

Stop after k steps

$$B(n) = 1 + \log_2 n$$



B(8) = 4

$$B(8) = 1 + 1 + 1 + 1 = 4$$

One path has one leaf; a branching tree has many calls at the same depth.

Repeated substitution exposes the same pattern

For $n \geq 8$, replace each recursive term by its recurrence again.

After j expansions, $0 \leq j \leq \log_2 n$

$$T(n) = 2^j T(n/2^j) + jn$$

Stop at the base case

$$j = \log_2 n$$

THE RECURRENCE

$$T(n) = 2T(n/2) + n$$

REPLACE $T(n/2)$

$$= 2[2T(n/4) + n/2] + n$$

DISTRIBUTE THE FACTOR

$$= 4T(n/4) + 2n$$

EXPAND ONCE MORE

$$= 8T(n/8) + 3n$$

Each expansion adds one whole level of work.

Repeated substitution discovers a candidate. Induction verifies it for every allowed input.

Substitution turns the candidate into a proof

Claim: $T(n) = n \log_2 n + cn$ for every power of two $n \geq 1$.

Base case: $n = 1$

$$1 \log_2 1 + c = c = T(1)$$

Induction hypothesis

$T(m) = m \log_2 m + cm$ **for smaller powers of two** m

The permitted substitution

Use the hypothesis at $m = n/2$, where $n \geq 2$.

The base case catches the missing leaf term: $n \log_2 n$ alone gives 0 at $n = 1$.

The induction step closes exactly

For a power of two $n \geq 2$, start from the recurrence.

Use the hypothesis at $n/2$

$$T(n/2) = \tfrac{n}{2}\log_2(n/2) + \tfrac{cn}{2}$$

Simplify the logarithm

$$\log_2(n/2) = \log_2 n - 1$$

START FROM THE RECURRENCE

$$T(n) = 2T(n/2) + n$$

SUBSTITUTE THE HYPOTHESIS

$$= 2\left[\tfrac{n}{2}\log_2(n/2) + \tfrac{cn}{2}\right] + n$$

EXPAND AND SIMPLIFY

$$= n\log_2 n - n + cn + n$$

RECOVER THE CLAIM

$$= n\log_2 n + cn$$

The $-n$ and $+n$ cancel; the leaf term cn remains.

The base case and the induction step together prove the formula.

EXERCISE 3

Use both methods on a new recurrence

$V(n) = 3V(n/3) + n$, $V(1) = 2$, with $n = 3^k$.

01 **Use a tree to find the total work.**

$k = \log_3 n$ **internal levels cost** n **each. Leaves cost** $2n$**.**

02 **State the candidate and check the base.**

$V(n) = n \log_3 n + 2n$**; at** $n = 1$ **it gives 2.**

03 **Substitute the hypothesis at** $n/3$**.**

$3[\frac{n}{3} \log_3(n/3) + \frac{2n}{3}] + n = n \log_3 n + 2n$.

Tree method: discover the total. Substitution method: prove the candidate.

EXERCISE 4

Hanoi removes one disk; merge sort halves the input

Move n disks between three pegs, one disk at a time. Never place a larger disk on a smaller one.

01 **Write the recurrence, including its base case.**

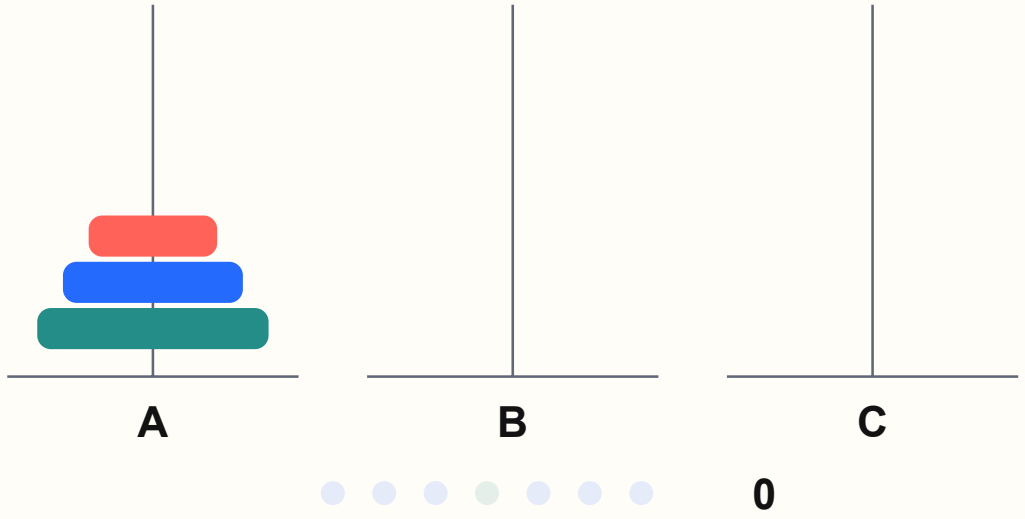
$h_n = 2h_{n-1} + 1, h_0 = 0.$

02 **Expand to find the number of moves.**

$h_n = \sum_{i=0}^{n-1} 2^i = 2^n - 1.$

03 **Compare with $T(n) = 2T(n/2) + n$.**

Hanoi: depth n, local work 1. Merge: depth $\log_2 n$, local work equal to the subproblem size.



$h_0 = 0$

Two recursive calls can lead to very different totals: check how much smaller each input becomes.

EXERCISE 4

Hanoi removes one disk; merge sort halves the input

Move n disks between three pegs, one disk at a time. Never place a larger disk on a smaller one.

01 **Write the recurrence, including its base case.**

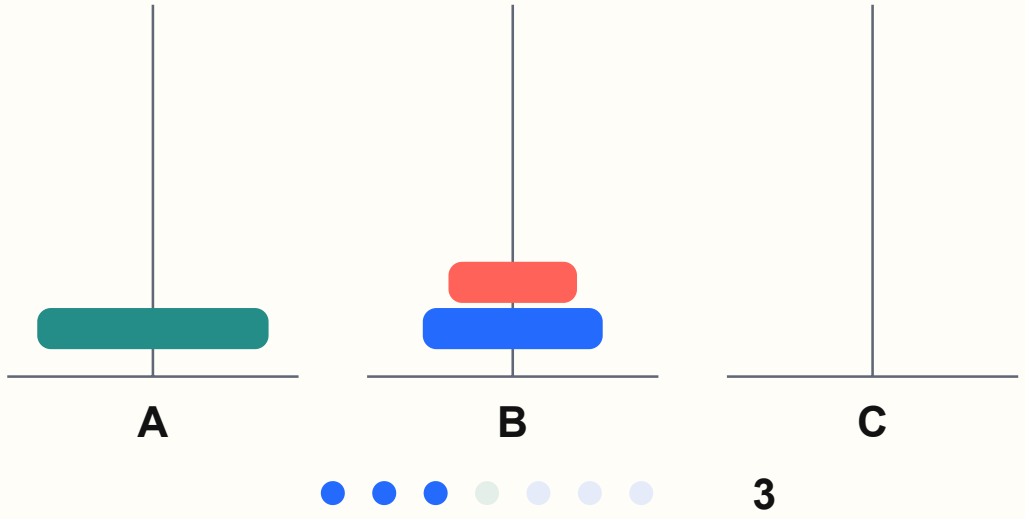
$h_n = 2h_{n-1} + 1, h_0 = 0.$

02 **Expand to find the number of moves.**

$h_n = \sum_{i=0}^{n-1} 2^i = 2^n - 1.$

03 **Compare with $T(n) = 2T(n/2) + n$.**

Hanoi: depth n, local work 1. Merge: depth $\log_2 n$, local work equal to the subproblem size.



$h_2 = 3$

Two recursive calls can lead to very different totals: check how much smaller each input becomes.

EXERCISE 4

Hanoi removes one disk; merge sort halves the input

Move n disks between three pegs, one disk at a time. Never place a larger disk on a smaller one.

01 **Write the recurrence, including its base case.**

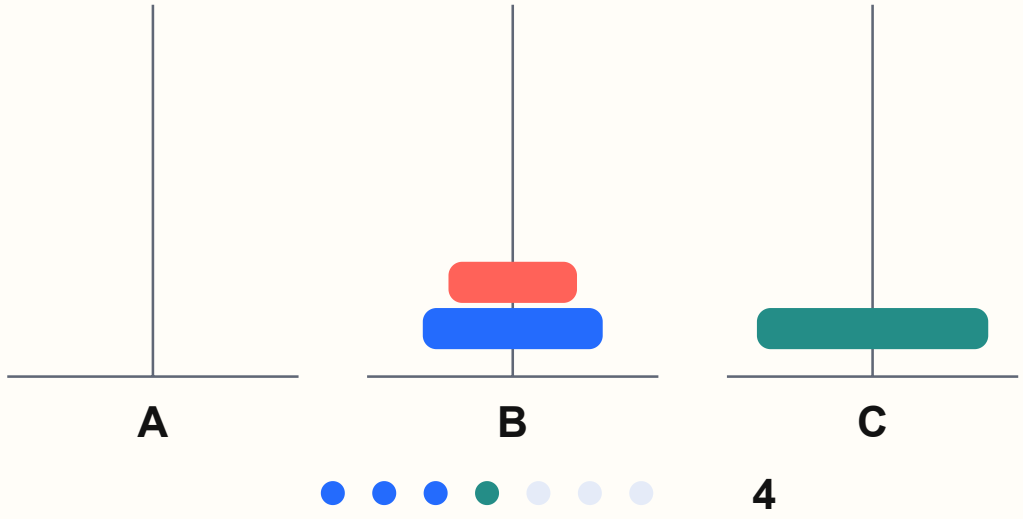
$h_n = 2h_{n-1} + 1, h_0 = 0.$

02 **Expand to find the number of moves.**

$h_n = \sum_{i=0}^{n-1} 2^i = 2^n - 1.$

03 **Compare with $T(n) = 2T(n/2) + n$.**

Hanoi: depth n, local work 1. Merge: depth $\log_2 n$, local work equal to the subproblem size.



$h_3 = h_2 + 1 + h_2$

Two recursive calls can lead to very different totals: check how much smaller each input becomes.

EXERCISE 4

Hanoi removes one disk; merge sort halves the input

Move n disks between three pegs, one disk at a time. Never place a larger disk on a smaller one.

01 **Write the recurrence, including its base case.**

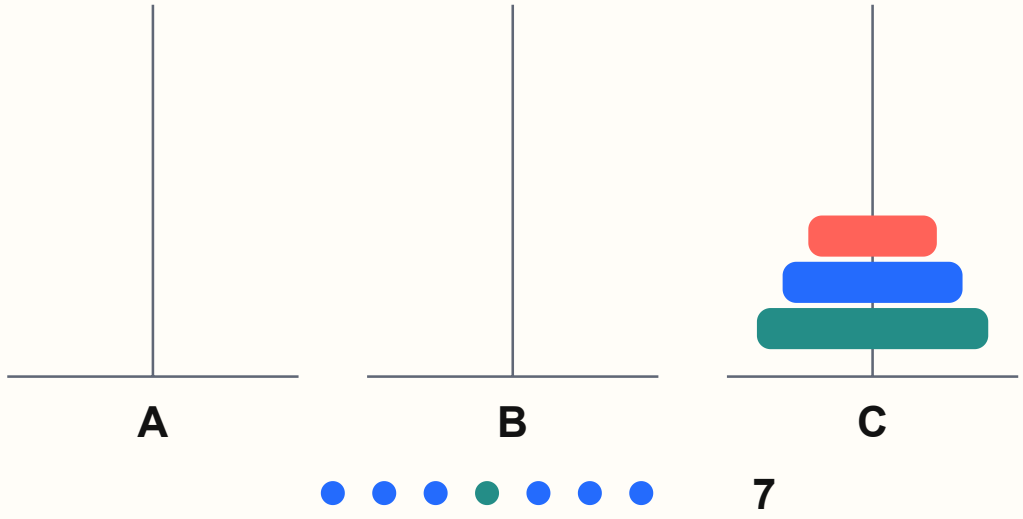
$h_n = 2h_{n-1} + 1, h_0 = 0.$

02 **Expand to find the number of moves.**

$h_n = \sum_{i=0}^{n-1} 2^i = 2^n - 1.$

03 **Compare with $T(n) = 2T(n/2) + n$.**

Hanoi: depth n, local work 1. Merge: depth $\log_2 n$, local work equal to the subproblem size.



$h_3 = 3 + 1 + 3 = 7$

Two recursive calls can lead to very different totals: check how much smaller each input becomes.

TAKEAWAY

A tree suggests the formula. Substitution proves it.

For $n = 2^k$ with $T(1) = c$, count work at internal levels and leaves. Check the base case, then substitute the smaller-case hypothesis.

$$T(n) = 2T(n/2) + n \quad \Longrightarrow \quad T(n) = n \log_2 n + cn$$